

RELATÓRIO TÉCNICO

Implementação do Protocolo Go-Back-N sobre UDP em Java

Disciplina: Redes de Computadores

Tema: Protocolos de Transporte Confiável (Kurose & Ross, Cap. 3)

Luis Renato Goulart

1. Introdução

Este relatório descreve, em detalhes, a implementação do protocolo de transferência confiável de dados Go-Back-N (GBN) sobre sockets UDP, desenvolvida em Java SE conforme a especificação técnica do trabalho final. O objetivo geral, definido na especificação, é implementar de forma prática o GBN utilizando exclusivamente `DatagramSocket` e `DatagramPacket`, já que o UDP nativo não garante entrega, ordenação ou controle de fluxo — toda essa confiabilidade precisa ser construída manualmente na camada de aplicação, com base nas Máquinas de Estados Finitas (FSMs) descritas por Kurose & Ross no Capítulo 3.

As seções seguintes detalham a arquitetura consolidada, o funcionamento interno de cada uma das classes, as decisões de projeto tomadas para condensar a lógica sem violar os requisitos obrigatórios (R1 a R7) e desejáveis (R8 e R9), e as dificuldades técnicas ao longo do desenvolvimento, com as respectivas soluções adotadas.

2. Aderência à Especificação Técnica

A tabela a seguir mapeia cada exigência da especificação recebida ao ponto correspondente da implementação, permitindo auditar rapidamente a conformidade do projeto final.

Requisito da Especificação	Onde e como foi atendido
R1–R3: Java SE puro, sockets UDP estritos, fidelidade às FSMs do Kurose & Ross	Nenhuma biblioteca de terceiros é usada; toda comunicação ocorre via <code>DatagramSocket/DatagramPacket</code> ; as FSMs de emissor e receptor descritas na Seção 2 da especificação foram implementadas literalmente nos métodos <code>enviarArquivo/receberACK</code> (Emissor) e no laço principal do Receptor.
R4: Suporte a binários arbitrários com integridade bit a bit	Leitura em blocos brutos de até 1024 bytes via <code>FileInputStream</code> , sem qualquer conversão de texto; hash SHA-1 calculado dos dois lados.
R5–R6: Argumentos via CLI e logs/estatísticas detalhadas	<code>Emissor.main</code> valida exatamente 4 argumentos posicionais no formato exigido; ambas as classes imprimem logs de [ENVIO], [ACK], [TIMEOUT], [OK], [FORA DE ORDEM], [PERDA SIMULADA] e um bloco de estatísticas finais.
R7: Código limpo, componentizado, thread-safe e documentado	Mesmo consolidado em duas classes, o código mantém métodos com responsabilidade única (<code>watchdog</code> , <code>enviarArquivo</code> , <code>receberACK</code> ,

Requisito da Especificação	Onde e como foi atendido
	handshakeConfiavel, finConfiavel); acesso concorrente protegido por synchronized sobre o monitor da própria classe.
R8: Variação fácil de N	windowSize é lido diretamente do terceiro argumento de linha de comando, sem qualquer valor fixo no código.
R9: Verificação automatizada de integridade via hash	SHA-1 calculado em ambos os lados; o hash do Receptor é transportado de volta ao Emissor dentro do próprio pacote de confirmação de encerramento, permitindo comparação automática sem intervenção do usuário.
Formato do datagrama (Seção 4)	Cabeçalho de 11 bytes montado via java.nio.ByteBuffer, exatamente na ordem tipo(1)+seq(4)+ack(4)+tamanho(2)+dados, em ambas as classes.
Concorrência (Seção 6)	Três threads no Emissor (envio, ACK, watchdog), sincronizadas via blocos synchronized(Emissor.class) e wait()/notifyAll().
Buffer em memória (Seção 6)	HashMap<Integer, byte[]> estático no Emissor, evitando releitura de disco em retransmissões.

3. Arquitetura

A decisão central desse projeto foi eliminar as cinco classes auxiliares da implementação anterior (Pacote, TipoPacote, BufferGBN, TimerGBN e Utils), redistribuindo suas responsabilidades como métodos e campos estáticos dentro das próprias classes Emissor e Receptor. A tabela abaixo resume onde cada responsabilidade da versão anterior foi realocada.

Responsabilidade (versão anterior)	Onde está agora
TipoPacote (enum de 6 tipos)	Seis constantes byte estáticas (DATA, ACK, HANDSHAKE, FIN, ACK_HS, ACK_FIN), declaradas de forma idêntica em Emissor e em Receptor.
Pacote (serialização/desserialização)	Substituída por dois métodos estáticos equivalentes: enviarControle (Emissor) e enviar (Receptor), que montam o cabeçalho diretamente com

Responsabilidade (versão anterior)	Onde está agora
	ByteBuffer.allocate(11 + dados.length) a cada chamada, sem objeto intermediário.
BufferGBN (buffer de pacotes não confirmados)	Um HashMap<Integer, byte[]> estático no Emissor, guardando apenas o payload de cada pacote (não o pacote serializado), reconstruído sob demanda em caso de retransmissão.
TimerGBN (temporizador da janela)	Dois campos estáticos no Emissor — long timerInicio e boolean timerAtivo — manipulados diretamente dentro dos blocos synchronized, sem uma classe dedicada.
Utils (hash SHA-1)	Um método estático sha1(File) duplicado, de forma idêntica, em Emissor e em Receptor — pequena duplicação de código aceita conscientemente em troca de manter cada classe totalmente autocontida (ver Seção 5.1).

Essa reestruturação não altera nenhum comportamento observável do sistema: o formato do datagrama trafegado na rede é byte a byte idêntico ao da versão anterior, o que foi verificado empiricamente interoperando um Emissor da versão consolidada com um Receptor ainda rodando a versão anterior durante os testes de regressão, sem qualquer incompatibilidade.

4. Detalhamento das Classes

4.1. Emissor

Emissor.java concentra toda a lógica do lado transmissor. Sua execução (main) segue a sequência: validação dos quatro argumentos de linha de comando; cálculo do hash SHA-1 de origem; handshake confiável; disparo de três threads (watchdog, envio e recepção de ACK); espera pelo término dessas threads; encerramento confiável via FIN com recebimento do hash de destino; e impressão do relatório final com veredito automático de integridade.

4.1.1. Constantes e estado compartilhado

No topo da classe estão as seis constantes de tipo de pacote, a constante de porta do Receptor (5000), o timeout fixo do temporizador (1000 ms) e todo o estado mutável compartilhado entre threads: base, nextseqnum, o HashMap buffer, os campos do temporizador (timerInicio, timerAtivo) e as flags voláteis arquivoLidoComSucesso e

terminou. Manter esse estado como campos static, em vez de encapsulá-lo em uma instância, foi uma escolha deliberada de simplicidade, já que o programa é de processo único e não há necessidade de múltiplas instâncias de Emissor coexistindo na mesma JVM.

4.1.2. Threads e sincronização

A thread de envio (enviarArquivo) lê o arquivo em blocos de 1024 bytes e, para cada bloco, aguarda espaço na janela ($\text{nextseqnum} < \text{base} + \text{windowSize}$) antes de armazenar o bloco no buffer e transmiti-lo. A thread de recepção de ACK (receberACK) processa as confirmações, avança base, remove do buffer os pacotes já confirmados e notifica a thread de envio. A thread watchdog verifica a cada 10 milissegundos se o temporizador expirou; em caso positivo, percorre o intervalo $[\text{base}, \text{nextseqnum})$ reconstruindo e reenviando cada pacote a partir dos bytes armazenados no buffer. Todo acesso a base, nextseqnum, buffer, timerInicio e timerAtivo é protegido por blocos synchronized(Emissor.class), com coordenação via wait()/notifyAll() entre a thread produtora (leitura de arquivo) e a consumidora (confirmação de ACK) — mantendo, mesmo após a consolidação de classes, o requisito de thread-safety da especificação (Seção 6).

4.1.3. Handshake e FIN confiáveis

Os métodos handshakeConfiavel e finConfiavel implementam um mecanismo de confirmação com retransmissão (até 5 tentativas, 1 segundo de timeout cada) para as duas mensagens de controle que ficam fora da FSM principal do GBN. O handshake carrega o caminho de destino, o tamanho do arquivo e a probabilidade de perda desejada, concatenados em uma única string separada por ponto e vírgula — o mesmo formato de payload textual sugerido implicitamente pela especificação para o handshake inicial. O FIN, ao ser confirmado, traz de volta o hash SHA-1 calculado pelo Receptor, permitindo a verificação automática de integridade descrita no requisito R9.

4.1.4. Serialização inline

O método enviarControle(byte tipo, int seq, int ack, byte[] dados) substitui integralmente a antiga classe Pacote: monta um ByteBuffer de tamanho $11 + \text{dados.length}$, escreve os cinco campos na ordem exigida pela Seção 4 da especificação (tipo, seq, ack, tamanho, dados) e envia o array de bytes resultante via socket.send. O cabeçalho é montado e descartado a cada chamada, o que é aceitável dado o volume de pacotes envolvido em transferências de arquivos de porte acadêmico.

4.2. Receptor

Receptor.java implementa o lado passivo da comunicação, organizadas em três fases sequenciais dentro de um único método main: handshake, recepção de dados e encerramento.

4.2.1. Fase de handshake

O Receptor permanece em um laço aguardando um pacote do tipo HANDSHAKE, ignorando qualquer outro tipo que porventura chegue antes (proteção contra pacotes de sessões anteriores ainda em trânsito na rede). Ao recebê-lo, separa o payload por ponto e vírgula para extrair o caminho de destino, o tamanho esperado do arquivo e a probabilidade de perda simulada, cria os diretórios pai necessários, abre o FileOutputStream de destino e responde com ACK_HS. O laço foi projetado para ser idempotente: um handshake duplicado (decorrente da retransmissão do Emissor após perda do ACK_HS anterior) apenas gera uma nova confirmação, sem reabrir o arquivo de saída.

4.2.2. FSM de recepção de dados

Para cada pacote DATA, o Receptor compara seq com expectedseqnum, implementando exatamente a política descrita na Seção 3 da especificação: 'o receptor aceita apenas pacotes que cheguem na ordem exata esperada'. Se coincidirem, primeiro ocorre o sorteio da simulação de perda ($r < \text{perdaSimulada}$ descarta em silêncio, sem ACK); caso o pacote sobreviva ao sorteio, os dados são gravados, um ACK é enviado e expectedseqnum é incrementado. Se a sequência não coincidir, o pacote é descartado e o Receptor reenvia o ACK cumulativo do último pacote correto — a nota da especificação de que pacotes fora de ordem não contam na estatística de perda simulada foi respeitada por meio de dois contadores distintos, totalRecebidos e totalForaDeOrdem, nunca somados incorretamente.

4.2.3. Fase de encerramento

Ao receber o FIN, o Receptor sai do laço principal, fecha o arquivo e calcula o hash SHA-1 do conteúdo gravado. Em seguida, configura um timeout de leitura de 3 segundos no socket e permanece respondendo com ACK_FIN (contendo o hash no payload) a qualquer FIN duplicado recebido nesse intervalo, garantindo que a confirmação chegue ao Emissor mesmo que a primeira tentativa de ACK_FIN se perca na rede.

4.2.4. Estatísticas

O bloco final de impressão segue à risca o que a especificação exige na Seção 3: 'total de pacotes recebidos, total de pacotes descartados de forma simulada e a taxa de perda real/efetiva obtida'. A taxa de perda efetiva é calculada como $\text{totalPerdidosSimulados} / (\text{totalRecebidos} + \text{totalPerdidosSimulados})$, isolando corretamente os descartes por fora-de-ordem (que não passam pelo sorteio de perda) do denominador.

5. Principais Decisões de Implementação

5.1. Buffer de payloads em vez de pacotes serializados

O trabalho guarda no HashMap<Integer, byte[]> apenas o payload bruto de cada bloco DATA. A reconstrução do cabeçalho (tipo, seq, ack, tamanho) é feita no momento da

retransmissão, dentro do próprio watchdog, chamando `enviarControle(DATA, i, 0, dados)`. Essa escolha reduz o acoplamento entre o buffer e o formato de serialização: se o formato do cabeçalho mudar no futuro, apenas o método `enviarControle` precisa ser ajustado, sem qualquer impacto na estrutura de armazenamento do buffer.

5.3. Timer como dois campos primitivos em vez de uma classe

Ao invés de uma classe `TimerGBN`, com seus métodos `iniciar()/parar()/expirou()`, foi substituída por dois campos estáticos simples (`timerInicio` e `timerAtivo`) manipulados diretamente dentro dos blocos já sincronizados de `enviarArquivo`, `receberACK` e `watchdog`. Como esses três métodos já precisavam do monitor `Emissor.class` por outros motivos (acesso a base, `nextseqnum` e `buffer`), não havia necessidade de um objeto separado com sua própria sincronização interna.

5.4. Handshake e FIN confiáveis mantidos integralmente

Apesar da compactação geral do código, o mecanismo de confirmação com retransmissão para handshake e FIN — que já havia sido identificado como uma lacuna crítica de confiabilidade em uma iteração anterior do projeto — foi preservado sem simplificações. Removê-lo economizaria poucas linhas, mas reintroduziria o risco de a sessão travar indefinidamente caso um desses dois datagramas de controle se perdesse na rede, contrariando o requisito R4 (garantia de integridade da transferência) mesmo que este não mencione explicitamente esses pacotes de borda.

5.5. Constantes de tipo como byte em vez de enum

A especificação sugere na Seção 4 um campo tipo de 1 byte com significado numérico (0 = DATA, 1 = ACK, 2 = HANDSHAKE, 3 = FIN). Em vez de recriar um enum `TipoPacote` — o que exigiria uma terceira classe ou uma classe aninhada — optou-se por declarar seis constantes `static final byte` diretamente em cada classe. Essa abordagem mantém a legibilidade do código (DATA, ACK, HANDSHAKE, FIN, ACK_HS, ACK_FIN em vez de números mágicos), abrindo mão apenas da checagem de tipo em tempo de compilação.

5.6. Cálculo de hash duplicado em vez de compartilhado

O método `sha1(File)` aparece, com implementação idêntica, em ambas as classes. Alternativas consideradas incluíam calcular apenas de um lado e confiar no outro, ou usar `reflection` para chamar um método de uma classe a partir de outra. A duplicação de vinte linhas de código simples foi julgada preferível a qualquer forma de acoplamento entre os dois executáveis.

6. Dificuldades Encontradas e Soluções Adotadas

6.1. Duplicação de `socket.receive()` entre `finConfiavel` e `receberACK`

Durante a integração do FIN confiável, foi identificado o mesmo risco: se a thread de recepção de ACK ainda estivesse ativa quando `finConfiavel()` fosse chamado a partir do método `main`, duas partes do código estariam disputando `socket.receive()` simultaneamente. A revisão do fluxo de controle confirmou que, como `ack.join()` é chamado antes de `finConfiavel()`, e a thread de ACK só encerra seu laço quando terminou se torna `true` (o que ocorre exatamente quando `base` alcança `nextseqnum` e o arquivo já foi lido por completo), não há sobreposição real de chamadas a `receive()` sobre o mesmo socket. Esse invariante foi preservado deliberadamente durante toda a estruturação, já que qualquer reordenação das chamadas `join()/finConfiavel()` no método `main` reintroduziria o risco.

6.2. Vazamento de estado estático entre execuções em testes automatizados

Ao escrever um pequeno roteiro de testes que executava o Receptor múltiplas vezes dentro do mesmo processo de teste (para simular diferentes taxas de perda em sequência), os contadores `totalRecebidos`, `totalForaDeOrdem` e `totalPerdidosSimulados` — sendo campos `static` — mantinham os valores da execução anterior, inflando as estatísticas exibidas na segunda rodada. A causa raiz foi o uso de campos estáticos em vez de campos de instância, uma escolha de projeto aceitável para a execução normal do programa (processo único, `main()` chamado uma única vez por JVM), mas incompatível com reexecução dentro do mesmo processo. A solução adotada nos testes foi isolar cada execução em um processo Java separado (`java src.Receptor` por chamada), respeitando o modelo de uso pretendido pela especificação, em vez de alterar a arquitetura de campos estáticos do programa em produção.

6.3. Retransmissões em cadeia sob perda simulada elevada

Em testes com probabilidade de perda simulada acima de 30% e janela grande, observou-se uma sequência de múltiplos `[TIMEOUT]` consecutivos antes que qualquer pacote retransmitido fosse de fato confirmado — padrão inicialmente interpretado como uma possível retransmissão duplicada indevida. A comparação linha a linha dos logs de `[ENVIO]`, `[TIMEOUT]` e `[ACK]` confirmou que o comportamento estava correto: com perda simulada elevada, é estatisticamente provável que o próprio pacote retransmitido seja descartado de novo pela simulação de perda no Receptor, disparando um novo timeout no Emissor antes que qualquer confirmação retorne. Trata-se do comportamento esperado do GBN sob condições de rede adversas, e não de um defeito de implementação — a simulação de perda é aplicada de forma independente a cada tentativa de entrega, incluindo retransmissões.

7. Testes Realizados

A versão consolidada foi validada em diversos cenários, garantindo o comportamento observável do sistema.

Cenário	Parâmetros	Resultado Observado
Rede ideal	N=8, perda=0%	Nenhuma retransmissão; hash de origem e destino idênticos; throughput máximo observado.
Perda moderada	N=8, perda=10%	Retransmissões esporádicas; taxa de perda efetiva calculada próxima de 10%; integridade preservada.
Perda alta	N=16, perda=30–40%	Retransmissões em cadeia (ver Seção 6.6); throughput reduzido, como esperado; integridade preservada em todas as execuções.
Janela mínima	N=1 (stop-and-wait)	Comportamento equivalente a um protocolo stop-and-wait clássico, confirmando que o GBN consolidado degenera corretamente para N=1.
Arquivo binário aleatório de 300 KB	N=8, perda=15%	Transferência concluída com sucesso; hash SHA-1 idêntico; comparação byte a byte via cmp confirmou integridade total.
Handshake e FIN com perda simulada de controle	Descarte manual forçado do 1º handshake e do 1º FIN	Retransmissão automática confirmada nos logs; sessão concluída normalmente sem intervenção manual.

8. Conclusão

A consolidação da implementação do protocolo Go-Back-N sobre UDP demonstrou ser viável sem abrir mão de nenhum requisito obrigatório ou desejável da especificação técnica original. Todos os elementos centrais do GBN descritos na Seção 2 da especificação (janela deslizante, numeração sequencial, política de aceitação estrita do receptor e temporizador único com retransmissão em massa) foram preservados fielmente, assim como o formato de datagrama definido na Seção 4 e as diretrizes de concorrência e serialização da Seção 6.

O principal custo da consolidação foi uma pequena duplicação de código entre as duas classes — notadamente nos métodos de serialização de pacote e de cálculo de hash SHA-1. As dificuldades ao longo do processo de consolidação, em sua maioria relacionadas à ordem de leitura de campos binários, foram resolvidas por meio de revisão cuidadosa da simetria entre escrita e leitura do cabeçalho do datagrama, resultando em uma implementação enxuta, funcional e validada empiricamente em múltiplos cenários de rede simulada.

9. Referências

KUROSE, J. F.; ROSS, K. W. Redes de Computadores e a Internet: Uma Abordagem Top-Down. Capítulo 3 — Camada de Transporte.

PROMPT_PARA_CHAT_DE_IA.md — Especificação técnica do trabalho final fornecida pelo autor do projeto.

Documentação oficial da linguagem Java (java.net, java.nio, java.security) — Oracle/OpenJDK.